

XRPL Lending Protocol · Developer Feedback

Hackathon DeVinci Blockchain × Ripple · 12 and 13 September 2026

CY-HACK

Track 1 · open-ended vault
XLS-65, XLS-66 and TokenEscrow

github.com/charlyppr/xrpl-lending

NETWORK

Custom Hackathon Devnet · `rippled`
3.4.0-rc1, hackathon branch ·
`LendingProtocolV1_1` and
`fixCleanup3_4_0` enabled

LIBRARY

`xrpl@4.6.0` · the 15 XLS-65/66 transaction
types are typed · no raw JSON

FIRST TRANSACTION

50 minutes, of which 45 on blocked ports
(finding 8)

VERIFICATION

143 of 143 cited hashes validated on the
ledger · docs checked against XLS-65,
XLS-66 and `xrpl.org` · library claims
against the `rippled` and `xrpl.js` source

#	Finding	Category	Checked against
1	<code>fixCleanup3_4_0</code> changes impairment timing and the <code>LoanSet</code> counterparty signature. <code>xrpl.org</code> does not list it.	docs, amendments	ledger, XLS-66, <code>rippled</code>
2	A late <code>LoanPay</code> without <code>tfLoanLatePayment</code> returns <code>tecEXPIRED</code> . The <code>xrpl.org</code> <code>LoanPay</code> reference does not list this failure.	docs, error codes	ledger, XLS-66, <code>xrpl.org</code>
3	The first-loss capital covered 0.5 % of a defaulted loan. The broker then withdrew the remaining cover.	design, naming	ledger, <code>xrpl.org</code>
4	<code>xrpl.org</code> omits the sole-holder exception to share value. <code>LossUnrealized</code> is absent from the node while zero.	docs, UX	ledger, XLS-65, <code>xrpl.org</code>
5	<code>loan_info</code> and <code>loan_broker_info</code> do not exist. <code>mpt_holders</code> needs <code>Clio</code> , which the devnet does not expose.	missing primitive	ledger, <code>xrpl.org</code>
6	Four result codes each cover situations that call for different actions.	error codes	ledger
7	The hackathon build reverts the V1.1 <code>LoanBrokerSet</code> restriction. The event brief does not mention it.	event docs	ledger, <code>rippled</code>
8	The devnet listens only on ports 51233 and 51234. A filtered port and a stalled node show the same signature.	infrastructure	network tests
9	Closing a vault to deposits requires setting <code>AssetsMaximum</code> to <code>AssetsTotal</code> .	missing primitive	ledger, <code>xrpl.org</code>
10	Vault shares carry <code>lsfMPTCanTrade</code> . <code>OfferCreate</code> with a share amount returns <code>temDISABLED</code> .	docs, MPT	ledger, <code>xrpl.org</code>

1 · `fixCleanup3_4_0` is enabled and not listed on `xrpl.org`

The feature command reports `fixCleanup3_4_0` as enabled on the devnet. The `xrpl.org` *Known Amendments* page does not list it. The amendment changes two lending behaviours.

Impairment. XLS-66, section 3.10.4.2, condition 9: with `fixCleanup3_4_0`, `tfLoanImpair` fails while `currentTime <= NextPaymentDueDate`. The `xrpl.org` tutorial *Manage a Loan* says a broker can “manually impair a loan before a payment due date passes”.

Time relative to <code>NextPaymentDueDate</code>	<code>tfLoanImpair</code>
31 s before	<code>tecT00_S00N</code>
13 s before	<code>tecT00_S00N</code>
5 s after	<code>tesSUCCESS</code>

5 s after: 5486020BA8FF227C58409042F96BDC07246FDACC3F9B0EF24749555E319E972E

Counterparty signature. `rippled` PR #8162: with `fixCleanup3_4_0`, `CounterpartySignature` uses the CPT signing prefix. In `xrpl@4.6.0`, `signLoanSetByCounterparty` signs with `encodeForSigning`, the previous prefix, and has no option to change it. Its `LoanSet` fails local checks with `Counterparty: Invalid signature`. The same `LoanSet` signed with `encodeForSigningCounterparty` from `ripple-binary-codec` returned `tesSUCCESS`.

23631C3610DA97926E8BF9FC5BC4E76E504C9DFB2683C928E825DE0138397B2B

2 · A late `LoanPay` without `tfLoanLatePayment` returns `tecEXPIRED`

A loan 10 s past `NextPaymentDueDate`, within its grace period, rejected four `LoanPay` transactions with `tecEXPIRED`: the instalment, the instalment plus `LatePaymentFee`, the full balance, and a payment with `tfLoanFullPayment`. The same payment with `tfLoanLatePayment` (262144) returned `tesSUCCESS`.

- XLS-66, section 3.11.4.2, condition 11 defines this failure. The `xrpl.org` `LoanPay` reference lists eight error codes, and `tecEXPIRED` is not among them.
- `xrpl.org` describes `tfLoanLatePayment` as “Indicates that the borrower is making a late loan payment”.
- The explorer shows `tfLoanImpair` and `tfLoanDefault` by name, and `tfLoanLatePayment` as `0x00040000`.

- No field of the `Loan` node changed during 85 s of lateness, and no field holds the accrued late interest. A payment of `ceil(PeriodicPayment) + LoanServiceFee + LatePaymentFee` returned `tecINSUFFICIENT_PAYMENT`. A payment of 2 765 001 drops returned `tesSUCCESS`, and 765 003 drops were taken.

 **Fail - TecEXPIRED** 

Status

This transaction failed with a status code of `tecEXPIRED`, and validated in ledger **67844** on September 12, 2026 at 2:33:42 PM UTC

Explorer. `LoanPay` of 1.030001 XRP without flags.

96C3870480E4CB0C9E3B925F2965347DA0815A486CDB4B49545928E3206B0B44

 **Success**

Flags

`0x00040000`

Explorer. `LoanPay`, same loan, same amount, with `tfLoanLatePayment`.

EFAD383F9B622357CE67CB80518D9F9F5FC27BE611D43F26FD6942713FB608F
overpayment: 9A7589384C5142703AFFACBB91B1E223EC734F7ACBE51272A87525FE4EE9CA8

3 · The first-loss capital covered 0.5 % of a defaulted loan

Both brokers used `CoverRateMinimum` 10 % and `CoverRateLiquidation` 5 %.

Vault	Loan	Cover	Taken	Loss	Share value
200 XRP	50	50	0.25	49.75	1.000 → 0.751
5 XRP	4	1	0.02	3.98	1.000 → 0.204

On both vaults the cover taken equals $\text{debt} \times 10\% \times 5\%$ to the drop. The `xrpl.org` Lending Protocol page gives this formula, with an example that covers 1 % of the debt. `CoverRateLiquidation` applies to the minimum cover ($\text{DebtTotal} \times \text{CoverRateMinimum}$), not to the defaulted amount. After a default, `DebtTotal` is zero and so is the cover floor.

Second vault, in ledger order:

UTC	Transaction	Result
14:23:50	<code>VaultWithdraw</code> , all shares	<code>tecINSUFFICIENT_FUNDS</code>
14:27:01	<code>LoanManage</code> <code>tfLoanDefault</code>	<code>tesSUCCESS</code>
14:27:11	<code>VaultWithdraw</code> , all shares, 1.02 XRP	<code>tesSUCCESS</code>
14:27:22	<code>LoanBrokerCoverWithdraw</code> , 0.98 XRP	<code>tesSUCCESS</code>

D91BA47E816FDA60F9BBAD7F100B383099D87021B08DE065E841B9E8E34AFE2D
3338BA629F8772CDEC07B6F714F2AF3A1ADC8E9364249A54B95CF372FDB2880
first vault default: 91F458E3E6244E1C3005345C5C83CDAFA0E8A1AF6E5587E100C302CA56743BC7

Success

Flags

tfLoanDefault

- It modified the AccountRoot node of **rsR4QZgAQzjCk3TdoT12duNL7bFCNTQAQJ**
 - Balance increased by $\times 0.02$ XRP from $\times 1.00$ XRP to $\times 1.02$ XRP
- It modified the AccountRoot node of **rbhF8rmZsXZfiacsNFX62psL9u3irY24H**
 - Balance decreased by $\times 0.02$ XRP from $\times 1.00$ XRP to $\times 0.98$ XRP

Explorer. LoanManage tfLoanDefault moves 0.02 XRP from the broker's pseudo-account (rbhF8...) to the vault's (rsR4Q...).

923F7D616B8094C82197506CF2867907A93067041E3990D24BD07A19035F73EAC

Success

Amount

$\times 0.98$

Date/Time (UTC)

9/12/2026, 2:27:22 PM

Explorer. LoanBrokerCoverWithdraw of the remaining cover, 21 s after the default.

68DD97D949602FD470E0FA80B2526C3FDCFA3449095F7A7879F4D4430845D45E

4 • Share value and LossUnrealized

XLS-65 computes the assets returned on redemption as $\text{shares} \times (\text{AssetsTotal} - \text{LossUnrealized}) / \text{total shares}$, and waives the LossUnrealized deduction when the withdrawer is the sole outstanding shareholder. The xrpl.org Single Asset Vault page gives the formula and does not mention the exception.

- The Vault node has no LossUnrealized field until an impairment writes one. The impairment below moved it from absent to "3000001", with AssetsTotal at "5000002".
- AssetsAvailable, AssetsMaximum, PaymentRemaining and TotalValueOutstanding are also absent from their node when zero.
- The explorer's Detailed tab shows "It modified a node with type Vault" without field values. LossUnrealized appears in the Raw tab once the node is expanded.

Success

- It modified a node with type Loan
- It modified a node with type Vault

AssetsAvailable: "2000001"

AssetsMaximum: "100000000"

AssetsTotal: "5000002"

Flags: 0

LEVersion: 1

LossUnrealized: "3000001"

Explorer. LoanManage tfLoanImpair: Detailed tab, then Raw tab with the Vault node expanded.

C678C1C0A10BE889124F40C03017038654B278E2D464FAC54784F55B35318FC

5 • Read commands

vault_info is a rippled command. loan_info and loan_broker_info return unknownCmd and have no documentation page. xrpl.org documents mpt_holders as Clio-only. The devnet endpoints serve rippled, where mpt_holders returns unknownCmd. A depositor's view takes five calls:

Call	Returns
vault_info	vault totals, share issuance and share flags
account_objects mptoken	the depositor's shares
account_objects on the owner	the broker, filtered on VaultID
account_objects loan	the loans, on the broker's pseudo-account
mpt_holders	unknownCmd

Six values are computed client-side: gross and net share value, position value, withdrawable amount, utilisation and broker debt capacity.

6 • Four result codes cover situations that call for different actions

Code	Situation	Hash
tecINSUFFICIENT_FUNDS	LoanSet above the cover limit	5DC3A6E0...4147618B
	LoanSet above the vault liquidity	B92A90F5...259F9CAA
	LoanBrokerCoverWithdraw below the floor	4FB4A2B0...262F1370
tecLIMIT_EXCEEDED	VaultDeposit above AssetsMaximum	132BAF7C...027B5B4F
	AssetsMaximum set below AssetsTotal	37053F15...A88C3DC6
	DebtMaximum set below DebtTotal	82C14ED4...748E4121
tecNO_PERMISSION	tfLoanImpair sent by the borrower	F0304194...5DE2C112
	tfLoanUnimpair on a loan that is not impaired	13975FD3...6AA24537
tecNO_AUTH	Payment of shares to an account without MPToken	789AAF7E...5FD512A4
	Payment of non-transferable shares	E9DE504D...B8BE5FD9

tecHAS_OBLIGATIONS names a single cause on VaultDelete, LoanBrokerDelete and LoanDelete.

Fail - TecINSUFFICIENT_FUNDS

Status

This transaction failed with a status code of **tecINSUFFICIENT_FUNDS**, and validated in ledger **68327** on September 12, 2026 at 2:57:52 PM UTC

Explorer. LoanSet above the cover limit.

5DC3A6E00F31DA82C69AB77669C64691BD71CAC49C169736F53D7C724147618B

Fail - TecINSUFFICIENT_FUNDS

Status

This transaction failed with a status code of **tecINSUFFICIENT_FUNDS**, and validated in ledger **64415** on September 12, 2026 at 11:42:11 AM UTC

Explorer. LoanSet above the vault liquidity. Same result code.

B92A90F59EBF79108B5414F61A12656F321EBC264E1BB186FCC14CAF259F9CAA

7 • The hackathon build reverts the V1.1 LoanBrokerSet restriction

The V1.1 documentation states "LoanBrokerSet is restricted on open-ended vaults". The feature command reports LendingProtocolV1_1 as enabled. LoanBrokerSet returned tesSUCCESS on an open-ended vault and on a closed-ended vault. rippled PR #8076 added the restriction. The commit Revert "fix: Reject open-ended vaults at LoanBrokerSet (#8076)" removed it from the lending hackathon branch on 10 September 2026. The other V1.1 rules tested (new fields, date validation, phase locks) are enforced. The event brief does not mention the revert.

open-ended 781F54B5E77A768F4C02A54E3C55902AA9F1AC96AA04037AB97CE93FFB50BDE4
closed-ended 1961BE21CC4011F50AA926E1494E9D51F94FC57B0E9C0C1B8F8ADF6FEDD09D40

8 • Ports 51233 and 51234

The documented endpoints use ports 51233 (WebSocket) and 51234 (JSON-RPC). No endpoint is documented on port 443. On a Wi-Fi network, nc -z succeeded on both ports while curl on 51234 returned 0 bytes, and portquiz.net:51234 timed out. The same signature appeared five times during the event: three occurrences were traced to the local network, two could not be attributed. The first cost 45 minutes. The xrpl.js Client uses WebSocket only. The faucet ignores the destination field and creates a new account.

9 · Closing a vault to deposits

The `xrpl.org` `Vault` entry and `XLS-65` define `AssetsMaximum: 0` as no cap. The `VaultSet` reference does not repeat it. `VaultSet` exposes `Data`, `AssetsMaximum` and `DomainID`, and no flag closes a vault to deposits. Setting `AssetsMaximum` equal to `AssetsTotal` blocks deposits: a 1 XRP deposit then returns `tecLIMIT_EXCEEDED`.

132BAF7C1B8208E695A453C0EE061BC5E3528FC97571589D8A2EA66F027B5B4F

Success

Assets Maximum0

Date/Time (UTC)9/12/2026, 5:22:50 PM

Explorer. `VaultSet` with `AssetsMaximum 0` on a vault capped at 5 XRP and holding 3.

A336D71E738AA27C2B449DE05A0FE5C8945733A53420B9B4D8390172404CC1E2

Success

Amount×4.00

Date/Time (UTC)9/12/2026, 5:22:52 PM

Explorer. Two seconds later, a 4 XRP `VaultDeposit` brings the same vault to 7 XRP.

D27E21CBB4C55F70963BEDB00FDB28A4C25B8A355619CAE85566CA63E25B956

10 · Vault shares carry `lsfMPTCanTrade`

`vault_info` returns the share issuance with `Flags: 56`, which is `lsfMPTCanEscrow (0x08)`, `lsfMPTCanTrade (0x10)` and `lsfMPTCanTransfer (0x20)`. `VaultCreate` has no field that sets these flags. The `xrpl.org` `MPTokenIssuance` reference describes `lsfMPTCanTrade` as allowing holders to “trade their balances using the XRP Ledger DEX or AMM”. On the devnet, an `OfferCreate` with a share amount returned `temDISABLED`. The feature list contains `MPTokensV1` and `DynamicMPT`, and no amendment for MPT trading.

What we ran

Step	Transaction	Hash
Open-ended vault	<code>VaultCreate</code>	A14CB64F...159502E4
Deposit, 25 XRP	<code>VaultDeposit</code>	98A6A315...FB772A0A
Loan broker	<code>LoanBrokerSet</code>	A4A619AA...6480C5A5
First-loss capital, 5 XRP	<code>LoanBrokerCoverDeposit</code>	9DEA76E2...30BF79BB
Origination and drawdown	<code>LoanSet</code>	7E3006E8...5F63BAB0
Repayment	<code>LoanPay</code>	41B2EAAE...0E8EDC8C
Withdrawal with yield	<code>VaultWithdraw</code>	53133F76...642895BE

All seven returned `tesSUCCESS`. All 15 `XLS-65/66` transaction types were submitted at least once, with a full `tfLoanImpair` then `tfLoanDefault` cycle.

Guardrails triggered on purpose

Transaction	Result	Hash
<code>LoanSet</code> of 5000 XRP, vault holding 25	<code>tecINSUFFICIENT_FUNDS</code>	B92A90F5...259F9CAA
<code>LoanPay</code> of 1 XRP, instalment 25.5 XRP	<code>tecINSUFFICIENT_PAYMENT</code>	77F708D6...1E0541B4
<code>LoanBrokerCoverWithdraw</code> below <code>CoverRateMinimum</code>	<code>tecINSUFFICIENT_FUNDS</code>	D91DEFCE...2595A60F
<code>LoanBrokerSet</code> by a non-owner of the vault	<code>tecNO_PERMISSION</code>	33742272...A0FA40A8
<code>VaultWithdraw</code> above unlent liquidity	<code>tecINSUFFICIENT_FUNDS</code>	FB47313D...D76DAAC4
Cover withdrawal above the floor (control)	<code>tesSUCCESS</code>	90BF019F...8C4279B2

Closed-ended vault, one transaction per phase

Phase	Transaction	Result	Hash
Subscription	<code>VaultDeposit 20 XRP</code>	<code>tesSUCCESS</code>	2F8B4C3A...3E25E149
Investment	<code>VaultDeposit 10 XRP</code>	<code>tecEXPIRED</code>	85D62B25...B42C92CA
Investment	<code>VaultWithdraw 5 XRP</code>	<code>tecT00_S00N</code>	C5449EC0...60386C5C
Redemption	<code>VaultWithdraw</code>	<code>tesSUCCESS</code>	F95FA55F...23F8D8E2

Secondary sale of vault shares with `TokenEscrow`

Step	Transaction	Hash
Buyer opts in to the share MPT	<code>MPTokenAuthorize</code>	79F48365...B32C5E31
Buyer locks the payment	<code>EscrowCreate</code>	1F29F998...E539FBF8
Seller locks the shares	<code>EscrowCreate</code>	795D6AFE...FB204099
Buyer claims the shares with the preimage	<code>EscrowFinish</code>	029559F0...1D26A179
Seller claims the payment with the same preimage	<code>EscrowFinish</code>	5F359ADB...50BEF991

Lending-related amendments enabled

`LendingProtocol`, `LendingProtocolV1_1`, `SingleAssetVault`, `MPTokensV1`, `DynamicMPT`, `TokenEscrow`, `fixTokenEscrowV1`, `fixCleanup3_2_0`, `fixCleanup3_4_0`, `BatchV1_1`, `PermissionedDomains`, `Credentials`.

What worked

- `tecHAS_OBLIGATIONS` names its cause on the three delete transactions.
- `VaultDelete` removes third-party `MPToken` objects and re-funds their reserve.
- The cover floor is enforced to the drop. `CoverRateMinimum`, `CoverRateLiquidation` and `ManagementFeeRate` return `temINVALID` when changed on an existing broker.
- The closed-ended vault phases behave as `XLS-65` describes: deposits accepted then `tecEXPIRED`, withdrawals `tecT00_S00N` then accepted.

Limits

XRP only, so neither clawback transaction was tested with an issued asset. Payment schedules of 60 to 90 seconds. One build: the hackathon branch of `rippled 3.4.0-rc1`.

Material in the repo

- `scripts/raw-submit.mjs`: a `LoanSet` counterparty signer using `encodeForSigningCounterparty`.
- `scripts/_probe-audit-hashes.mjs`: checks every hash cited in a document against the ledger, and separates transaction hashes from ledger object IDs.
- `deck/capture-explorer.mjs`: the explorer screenshots in this PDF.
- Observed in passing: the Lending Protocol concept page spells `PrincipleOutstanding` and `depositor`. `validateVaultCreate` in `xrpl@4.6.0` accepts any number for `WithdrawalPolicy`; the values 0, 2, 3, 99 and 255 reach the network and return `temMALFORMED`.